

2026 年 4 月 27 日 Version x.x

ハッカソン ～計画書と設計書～

チーム 2

24G1146 : 川又 快斗

24G1053 : 熊本 朱純

24G1133 : 安尾 晃貴

25G1084 : 陳 夏薇

25G1089 : 常世田 隆一

25G1110 : 早川 和輝

25G1110 : 福島 健一

目次

第 1 章	プロジェクトの計画書	1
1.1	プロジェクトの概要	1
1.1.1	プロジェクトの題目	1
1.1.2	プロジェクトの目的	1
1.1.3	既存の技術とプロジェクトの独自性	2
1.2	スコープ・マネジメント	2
1.2.1	成果物	2
1.2.2	プロジェクトの対象範囲と非対象範囲	2
1.2.3	機能分解構造 FBS	2
1.2.4	製品分解構造 PBS	2
1.2.5	FBS と PBS の対応関係	2
1.3	作業計画	2
1.3.1	作業分解構造 WBS と管理番号	2
1.3.2	アローダイアグラムとクリティカルパス	2
1.3.3	役割分担	2
1.3.4	ガントチャート	2
1.4	検証と妥当性確認: V&V 計画	2
1.4.1	プロジェクトの成果物に対する有効指標 MOE	3
1.4.2	有効指標 MOE に対応する性能指標 MOP	3
1.4.3	システムテストで評価する性能指標 MOP	4
1.4.4	結合テストで評価する性能指標 MOP	4
1.4.5	単体テストで評価する性能指標 MOP	5
1.4.6	プロジェクト中に監視する技術性能指標 TPM	5
1.4.7	プロジェクト中に監視する指標 TPM (リソース監視)	6
1.5	資源・調達管理	6
1.6	リスク管理	7
第 2 章	システムの基本設計	9

2.1	機能一覧	9
2.2	システム構成図	9
2.3	操作インターフェース設計	10
2.4	状態遷移図	10
第 3 章	システムの詳細設計	13
3.1	部品一覧	13
3.2	ハードウェア設計	14
3.2.1	センサー制御モジュールの設計	14
3.2.2	同期通信ネットワークの構築	14
3.3	ソフトウェア設計	14
3.3.1	センサー制御ユニットの設計	14
3.3.2	同期制御ユニットの設計	15
3.3.3	楽器制御ユニットの設計	15
3.3.4	ファイル構成	15
3.3.5	オブジェクト設計（クラス図）	17
3.3.6	関数設計	17
3.4	処理フローの設計	18
3.5	テストの設計	19
3.5.1	センサー制御ユニットのテスト設計	19
3.5.2	同期制御ユニットのテスト設計	19
3.5.3	楽器制御ユニットのテスト設計	19

第 1 章 プロジェクトの計画書

1.1 プロジェクトの概要

1.1.1 プロジェクトの題目

1.1.2 プロジェクトの目的

プロジェクトの目的と最終的なゴール，解決すべき課題を記述する．

指揮動作に応じて複数の Arduino がリアルタイムに同期し，可変テンポで演奏および視覚表現を行うシステム作成する．

1.1.3 既存の技術とプロジェクトの独自性

1.2 スコープ・マネジメント

1.2.1 成果物

1.2.2 プロジェクトの対象範囲と非対象範囲

1.2.3 機能分解構造 FBS

1.2.4 製品分解構造 PBS

1.2.5 FBS と PBS の対応関係

1.3 作業計画

1.3.1 作業分解構造 WBS と管理番号

1.3.2 アローダイアグラムとクリティカルパス

1.3.3 役割分担

1.3.4 ガントチャート

1.4 検証と妥当性確認: V&V 計画

本節では、開発する自動合奏システムがユーザーの要求と目的を満たしているか（妥当性確認: Validation）、およびシステムが設計通りに正しく構築されているか（検証: Verification）を評価するための V&V 計画について述べる。

1.4.1 プロジェクトの成果物に対する有効指標 MOE

有効指標 MOE (Measures of Effectiveness) は、ユーザー（指揮者および聴衆）の観点から、システムが運用環境において目的を達成しているかを示す定性的な指標である。本システムでは以下の 3 点を MOE として定義する。

- ・ **MOE-1 指揮動作の即応性:** 指揮棒を振り下ろす動作に対して、システムの演奏が違和感なく即座に開始されること。
- ・ **MOE-2 テンポ制御の滑らかさ:** 距離センサーに対する手の接近・離反動作に対して、楽曲のテンポ (BPM) が演奏を破綻させることなく直感的かつ滑らかに変化する。
- ・ **MOE-3 合奏の同期性:** 4 台の楽器ユニット（リズム用 3 台、ビート用 1 台）が、人間が聴覚的にズレを感じない精度で調和して合奏を行うこと。

1.4.2 有効指標 MOE に対応する性能指標 MOP

前項の MOE を技術的かつ定量的に評価するため、システムの物理的・ソフトウェア的な性能指標である MOP (Measures of Performance) を以下のように定義する。

- ・ **MOP-1 エンドツーエンド応答時間 (MOE-1 に対応) :** 指揮棒の加速度センサーが設定閾値を超える急激な変化（拍の起点）を捉えてから、各楽器ユニットのアクチュエータが物理的に駆動を開始するまでの全体時間遅延。
- ・ **MOP-2 テンポ追従遅延 (MOE-2 に対応) :** 距離センサーで規定以上の変位を検出してから、全 4 台の楽器ユニットにおける BPM 変数が更新され、次拍の待機時間計算に反映されるまでの時間。
- ・ **MOP-3 ユニット間同期誤差 / ジッタ (MOE-3 に対応) :** 同期制御ユニットからの同一のブロードキャスト信号を受信した 4 台の楽器ユニット間で生じる、発音タイミングの最大の時間差（マイクロ秒～ミリ秒オーダー）。一般に人間の聴覚が和音のズレを認識する閾値（約 20 ミリ秒～30 ミリ秒）を大きく下回ることを目標とする。

1.4.3 システムテストで評価する性能指標 MOP

全 6 台のユニット（センサー制御 1 台，同期制御 1 台，楽器制御 4 台）を統合した最終形態において，システム全体としての MOP を評価する。

全体応答遅延の測定

ハイスピードカメラ等の外部計測器を用い，物理的な指揮の振り下ろし動作から発音アクチュエータが動くまでのエンドツーエンド応答時間（MOP-1）を計測し，目標値以内に収まっているかを評価する。

合奏同期誤差の観測

オシロスコープ等のロジックアナライザを用いて，4 台の楽器ユニットの出力ピン（駆動信号）を同時に観測する。演奏開始時およびテンポ変更時において，各ピンの立ち上がりエッジの時間差（MOP-3）が許容範囲内であることを評価する。

長時間稼働時の安定性

システムを一定の BPM で長時間稼働させ，メモリリークや処理落ちによる演奏の停止，およびテンポの不自然な揺らぎが発生しないかを評価する。

1.4.4 結合テストで評価する性能指標 MOP

サブシステム間におけるインターフェースの整合性と，通信ネットワークの信頼性を評価する。

ユニット間通信遅延の測定

「センサー制御から同期制御」，および「同期制御から各楽器制御」へのデータ通信にかかるレイテンシを測定する。この遅延が MOP-1 および MOP-2 のボトルネックになっていないかを評価する。

パケットロス・通信エラー率の評価

同期制御ユニットから 4 台の楽器ユニットに対する信号の一斉送信テストを規定回数（例：1000 回）実施する。外部ノイズ等の影響下においても，制御フラグや BPM データの欠損率（パケットロス率）がシステムのフェールセーフ基準を満たしているかを評価する。

1.4.5 単体テストで評価する性能指標 MOP

各マイクロコントローラ、センサー、およびアクチュエータ単体の機能と処理能力を評価する。

センサーサンプリングレートと判定精度

センサー制御ユニット単体において、加速度および距離センサーのデータを安定して取得できる更新周期 (Hz) を評価する。また、規定の指揮動作を行った際のトリガー生成の成功率 (真陽性率)、および静止時の誤検知率 (偽陽性率) を計測する。

時間計算誤差の評価

楽器制御ユニット単体において、指定された BPM 値から算出される「1 拍あたりの待機時間 (理論値)」と、マイコン内部のタイマー関数による「実際の待機時間 (実測値)」との誤差を評価し、テンポの正確性を検証する。

アクチュエータの物理的応答性

スピーカーやソレノイド等の発音アクチュエータ単体に対し、電気信号を与えてから実際に音響エネルギーが発生するまでの機械的な遅延を測定する。

1.4.6 プロジェクト中に監視する技術性能指標 TPM

技術性能指標 TPM (Technical Performance Measures) は、プロジェクトの進行に伴い、システムが最終的な性能要求 (MOP) を達成できるかを継続的に評価するための監視指標である。本システムでは、特に動的なパフォーマンスに関する以下の指標を定期的に計測し、性能の劣化傾向がないかを監視する。

通信レイテンシの推移

開発フェーズが進行し、通信するデータ量や処理が複雑化する過程において、同期制御ユニットから各楽器ユニットへの通信遅延が規定値 (例: 10 ミリ秒未満) を維持できているかを監視する。

センサー処理のループ実行時間

センサー制御ユニットにおけるノイズ除去 (平滑化フィルタ) や閾値判定アルゴリズムの計算コストが、要求されるサンプリングレート (例: 50Hz 以上) を阻害していないかを監視する。

1.4.7 プロジェクト中に監視する指標 TPM（リソース監視）

前項の動的パフォーマンスに加え、開発の進行によって枯渇するリスクのあるハードウェアリソースに関する指標（静的 TPM）を以下のように定義し、監視を行う。

SRAM およびフラッシュメモリ使用率

各マイクロコントローラ（特に楽曲データを配列として保持する楽器制御ユニット）におけるメモリ使用率をコンパイル時に毎回記録し、実装の追加によって上限容量（Arduino Uno の場合、SRAM 2KB / フラッシュ 32KB）を逼迫していないかを監視する。

システム全体の消費電力推移

各ユニットおよび発音アクチュエータ（スピーカー、ソレノイド等）が同時に駆動した際の最大消費電流を監視し、用意した電源容量（AC アダプタやモバイルバッテリー）の定格を超過する兆候がないかを評価する。

1.5 資源・調達管理

本節では、プロジェクトをスケジュール通りに完遂するために必要なハードウェア、ソフトウェア、および人的リソースの調達と管理計画について述べる。

- ・ **ハードウェア部品の調達計画:** 分散制御の核となる計 6 台のマイクロコントローラ、通信用モジュール、加速度および距離センサー、発音アクチュエータは、初期動作検証における破損リスク（予備部品）を考慮し、必要数に対し 10%～20%の余剰を持たせてプロジェクト初期段階で一括調達する。
- ・ **人的リソースと役割分担:** 開発メンバーのスキルセットに応じ、「センサー信号処理」「同期・通信プロトコル設計」「楽器制御・発音実装」の 3 つのサブシステムに担当を割り当てる。各担当者は自身のユニットの単体テストまでを責任範囲とし、結合テスト以降はチーム全体で連携してデバッグにあたる。
- ・ **開発・検証設備の確保:** ユニット間の通信波形や同期ズレ（ジッタ）をミリ秒単位で可視化するため、プロジェクト期間中を通じてオシロスコープやロジックアナライザ等の測定機器を常時利用できる環境を確保する。

1.6 リスク管理

プロジェクトの遂行およびシステムの運用における潜在的なリスクを特定し、それらが顕在化した際の影響を最小限に抑えるための対策（コンティンジェンシープラン）を以下に定義する。

リスク 1: 通信ノイズによる制御パケットの欠損

影響: 楽器ユニットが同期開始の合図を取りこぼし、合奏が破綻する。

対策: ソフトウェア制御によるパケットの再送要求、またはチェックサム（誤り検出符号）の導入によりデータの完全性を担保する。ハードウェア面では、通信線のシールド強化や適切なプルアップ抵抗の配置を実施する。

リスク 2: 環境光・音波によるセンサーの干渉

影響: 距離センサー（赤外線または超音波）が、舞台照明や他の音響機器と干渉し、意図しないテンポ変更（BPM の乱高下）が発生する。

対策: センサーに物理的なフード（遮光・指向性制限）を取り付けるとともに、ソフトウェア側で急激な外れ値を無視する異常値フィルタリングを実装する。

リスク 3: 楽曲データの肥大化によるメモリ不足

影響: 長時間の楽曲や複雑な和音を実装しようとした結果、楽器ユニットの SRAM を使い果たしシステムがクラッシュする。

対策: 楽曲データをプログラムメモリ領域（PROGMEM）に格納して SRAM の消費を回避する。それでも不足する場合は、SD カードモジュールを用いた外部ストレージからの逐次読み込み方式へ設計を変更する。

リスク 4: 部品調達の遅延・初期不良

影響: ハードウェアの組み立てが進まず、プロジェクト全体のスケジュールが遅延する。

対策: 資源・調達管理で定めた通り予備部品をあらかじめ確保する。さらに、ハードウェアが未完成の段階でも、PC 上のシリアルモックアップを用いてソフトウェアロジックの開発とテストを並行して進める（シミュレーション開発の導入）。

第 2 章 システムの基本設計

本章では、開発する自動合奏システムの全体像を明確にするため、システムの基本設計について述べる。機能分解構造（FBS: Function Breakdown Structure）に基づく機能一覧、計 6 台のマイクロコントローラを用いた分散型システム構成、直感的な操作インターフェース、およびシステムの状態遷移について定義する。

2.1 機能一覧

本システムは、指揮棒による動作入力と距離センサーによる空間入力を統合し、複数の楽器ユニットを同期制御するものである。機能分解構造（FBS）に基づき、本システムの主要な機能を以下のように定義する。

- ・ **動作入力検知機能:** 指揮棒に内蔵された加速度センサーを用い、演奏開始の合図となる特定の振り下ろし動作（拍の起点）を検知する。
- ・ **テンポ制御機能:** 距離センサーを用い、操作者の手などの物体との距離を計測することで、演奏中のテンポ（BPM）を動的に変化させる。
- ・ **分散型同期通信機能:** センサーの解析結果を同期制御ユニットに集約し、リズム用およびビート用の各楽器ユニットに対して低遅延で指示をブロードキャストする。
- ・ **パート別自動演奏機能:** 受信した同期信号に基づき、リズム用ユニット（和音・メロディ）とビート用ユニット（打楽器）が独立しつつも協調して楽曲を演奏する。

2.2 システム構成図

本システムは、入力モジュール、センサー制御ユニット（1 台）、同期制御ユニット（1 台）、および楽器制御ユニット群（リズム用 3 台、ビート用 1 台）による分散アーキテクチャを採用している。システム全体の構成要素を以下に示す（ここにシス

テム構成図の画像を挿入する)。

- ・ **入力デバイス部**: 指揮動作を取得する加速度センサー, およびテンポ制御用の距離センサー。
- ・ **センサー制御ユニット**: 入力デバイスからの生データを解析し, トリガー信号と BPM データに変換する役割を担う。
- ・ **同期制御ユニット**: センサー制御ユニットからデータを受け取り, 後段の全楽器ユニットへ一斉に制御指令を分配するハブの役割を担う。
- ・ **楽器出力ユニット群**: 同期制御ユニットからの指令を受信し, 独立して演奏を行う 4 台のスレーブユニット (リズム用 3 台, ビート用 1 台), およびそれぞれに接続された発音アクチュエータ (スピーカー, ソレノイド等)。

2.3 操作インターフェース設計

本システムの操作インターフェースは, 従来の音楽演奏における指揮者の動作を模倣し, 直感的かつ物理的に行えるよう設計した。

1. **演奏開始操作 (トリガー入力)**: ユーザーが加速度センサーを内蔵した指揮棒を振り下ろすことで, その急激な動きが演奏開始の合図として認識される。
2. **テンポ操作 (BPM 入力)**: 演奏中, ユーザーが操作卓に配置された距離センサーに対して手を近づける, あるいは遠ざける動作を行うことで, その計測距離に応じてリアルタイムに演奏速度を増減させる。

2.4 状態遷移図

本システムは主に「初期化状態」「待機状態」「演奏状態」の基本状態を持ち, 入力条件に応じて以下のように遷移して動作する (ここに状態遷移図の画像を挿入する)。

初期化状態から待機状態への遷移

システムへの電源投入後, 全 6 台のユニットがセンサーのキャリブレーションや楽曲データの計算等の初期設定を行い, 開始指示を待つ待機状態へと移行する。

待機状態から演奏状態への遷移

加速度センサーが指揮棒の振り下ろし動作を検知した場合、センサー制御ユニットから開始フラグが発行され、全楽器ユニットが同期して演奏を開始する。

演奏状態中のテンポ更新

演奏中に距離センサーの計測値に変動があった場合、新たな BPM 値が同期制御ユニットを経由して全楽器ユニットに送信され、演奏を中断することなく次拍からテンポが更新される。

演奏状態から待機状態への遷移

規定の楽曲データの終端に達し演奏が終了した場合、自動的に初期の待機状態へ戻り、次の指揮動作を受け付ける状態となる。

第 3 章 システムの詳細設計

本章では、前章で定義した基本設計に基づき、自動合奏システムを実際に構築するための詳細な設計について述べる。本システムは合計 6 台のマイクロコントローラを用いた分散制御アーキテクチャを採用している。製品分解構造（PBS: Product Breakdown Structure）に基づく部品の選定、ハードウェア構成、各ユニットのソフトウェア設計、および処理フローを明らかにする。

3.1 部品一覧

本システムを構成する主要なハードウェア要素を以下に定義する。

- ・ **センサー制御用マイクロコントローラ (1 台):** センサー情報の取得と解析を行い、物理的な動作をデータ化するユニット。
- ・ **同期制御用マイクロコントローラ (1 台):** センサー制御ユニットからデータを受け取り、システム全体の時間軸と各楽器への信号分配を管理する中枢ユニット。
- ・ **楽器制御用マイクロコントローラ【リズム用】 (3 台):** 和音やメロディなど、音階を伴うリズム演奏を独立して行うスレーブユニット。
- ・ **楽器制御用マイクロコントローラ【ビート用】 (1 台):** ドラミングなどの打楽器的なビート演奏を専任で行うスレーブユニット。
- ・ **加速度センサーモジュール:** 指揮棒に内蔵し、3 軸方向の加速度を計測する。
- ・ **距離センサーモジュール:** テンポ制御用として操作デバイスに組み込み、距離を計測する。
- ・ **発音アクチュエータ:** 各スレーブユニットに接続される圧電スピーカーや打楽器用ソレノイド等。
- ・ **配線・通信用部品:** ユニット間のシリアル通信や I2C 通信を確立するためのワイヤ類。

3.2 ハードウェア設計

本システムのハードウェアは、入力処理と同期分配処理を別々のマイクロコントローラに分離することで負荷を分散し、4 台の楽器ユニットを低遅延で同期させる構成をとる。

3.2.1 センサー制御モジュールの設計

指揮棒に搭載した加速度センサー、および操作卓に配置した距離センサーは、センサー制御用マイクロコントローラに接続される。本ユニットはセンサーの生データをリアルタイムで処理し、有線通信を通じて隣接する同期制御用マイクロコントローラへ結果のみを送信する設計とする。これにより、センサーのノイズ除去や複雑な浮動小数点演算が、他の制御の妨げになることを防ぐ。

3.2.2 同期通信ネットワークの構築

同期制御用マイクロコントローラをハブとし、スター型あるいはバス型の通信ネットワークを構築する。同期制御ユニットは、センサー制御ユニットから「指揮の振り下ろし（トリガー）」および「距離に応じたテンポ（BPM）」のデータを受信する。その後、3 台のリズム用ユニットと 1 台のビート用ユニットに対して、汎用非同期通信（UART）等を介して一斉に制御信号をブロードキャスト送信する構成とする。

3.3 ソフトウェア設計

本節では、役割の異なる 3 種類のユニット（センサー制御、同期制御、楽器制御）に実装されるソフトウェアの構造について述べる。

3.3.1 センサー制御ユニットの設計

センサー制御ユニットは、ハードウェアから入力される時系列データを解析し、物理的な事象をデジタルフラグへと変換する。

1. **動作検知アルゴリズム:** 加速度データの微小時間における変動量を算出し、設定した閾値を超過した時点を「拍の起点」と判定して同期制御ユニットへ送信

- する。
2. **テンポ算出アルゴリズム:** 距離センサーの計測値を平滑化し、60 から 180 BPM などのテンポ範囲へとマッピング変換し、連続的に同期制御ユニットへ送信する。

3.3.2 同期制御ユニットの設計

同期制御ユニットは、システム全体の指揮者として機能する。

1. **信号集約と配信先制御:** センサー制御ユニットから送られるフラグをシステム時間と同期させる。リズム用（3 台）とビート用（1 台）の役割の違いを考慮し、全体への一斉送信や、特定ユニットに対する休符・演奏切り替え等のルーティング処理を担う。
2. **動的テンポ指令のバッファリング:** テンポ変更の指示が連続して発生した際、通信帯域を圧迫しないよう適切な更新頻度に間引き、パケット化して送信する処理を行う。

3.3.3 楽器制御ユニットの設計

楽器制御ユニット（リズム用およびビート用）は、受信した制御信号に基づく精緻な時間制御を核とする。

1. **役割に応じた楽曲データの保持:** リズム用ユニットには音高と音価のペア配列を、ビート用ユニットには打楽器のタイミング配列を保持させる。
2. **テンポ追従型演奏ロジック:** 受信した BPM 情報をマイクロ秒単位の待機時間に変換する。四分音符 1 拍の長さ T [ms] は $T = 60000/\text{BPM}$ として算出され、この基準時間を元に演奏を継続する。

3.3.4 ファイル構成

ソフトウェアは、機能ごとに 3 つの独立したプロジェクト群として構成される。各プロジェクトのディレクトリ構造を以下に示す。

【センサー制御プロジェクト】

SensorProject/

```
|-- SensorMain.ino  
|-- SensorProcessor.h  
|-- SensorProcessor.cpp
```

SensorMain.ino

センサーの初期化とデータ取得のメインループ.

SensorProcessor (.h / .cpp)

平滑化处理, 拍の判定, および BPM 算出アルゴリズムをカプセル化したモジュール.

【同期制御プロジェクト】

SyncProject/

```
|-- SyncMain.ino  
|-- SyncController.h  
|-- SyncController.cpp
```

SyncMain.ino

通信の受信待機と, システム全体の時間軸管理を行うメインループ.

SyncController (.h / .cpp)

受信データの解析と, 4 台の楽器ユニットに向けたデータパケットの構築・送信モジュール.

【楽器制御プロジェクト (リズム用 / ビート用共通)】

InstrumentProject/

```
|-- InstrumentMain.ino  
|-- MusicData.h  
|-- InstrumentPlayer.h  
|-- InstrumentPlayer.cpp
```

InstrumentMain.ino

同期制御ユニットからの信号受信と、割り込みを用いた演奏管理.

MusicData.h

ユニットの役割（リズムまたはビート）に応じた楽曲データ配列の定義ファイル.

InstrumentPlayer (.h / .cpp)

BPM に基づく待機時間計算とハードウェア駆動を実装するモジュール.

3.3.5 オブジェクト設計（クラス図）

システムはオブジェクト指向設計に基づく。以下に主要なクラスの設計を示す（ここに UML クラス図の画像を挿入する）.

・ SensorProcessor クラス（センサー制御用）

- 主な属性: 加速度データ, 距離データ, 閾値.
- 主な操作: readSensors(), detectBeat(), calculateBPM().

・ SyncController クラス（同期制御用）

- 主な属性: 現在の BPM, 演奏状態フラグ.
- 主な操作: receiveFromSensor(), broadcastToInstruments().

・ InstrumentPlayer クラス（楽器制御用）

- 主な属性: 現在の演奏 BPM, 楽曲データ配列へのポインタ, 再生インデックス.
- 主な操作: receiveCommand(), updateTempo(), playNextNote().

3.3.6 関数設計

本節では、主要な関数の仕様を定義する.

SensorProcessor::detectBeat()

引数: なし / 戻り値: bool

処理内容: 加速度の変動量が設定した閾値を超過した場合に「拍の起点」と判定し true を返す.

SensorProcessor::calculateBPM()

引数: なし / 戻り値: int

処理内容: 距離センサーの計測値を平滑化し, 許容するテンポ範囲へと線形補間して BPM 値を返す.

SyncController::broadcastToInstruments(bool isStart, int bpm)

引数: bool isStart, int bpm / 戻り値: なし

処理内容: 開始フラグと BPM 情報をパケット化し, リズム用 3 台およびビート用 1 台の合計 4 台のユニットへ一斉送信する.

InstrumentPlayer::updateTempo(int currentBPM)

引数: int currentBPM / 戻り値: なし

処理内容: 受信した BPM 値に基づき, 1 拍あたりの実時間 T [ms] を算出し, 発音時間の基準クロックを更新する.

InstrumentPlayer::playNextNote()

引数: なし / 戻り値: なし

処理内容: 楽曲データ配列から情報を取得し, 算出した基準時間を元にハードウェアを駆動する.

3.4 処理フローの設計

システムの具体的な処理手順を時系列で述べる.

1. **【センサー制御】物理入力の検知:** 加速度および距離センサーからデータを取得し, 急激な変化があった場合にトリガー信号を, 距離変化があった場合に BPM データを同期制御ユニットへ送信する.
2. **【同期制御】指令の分配:** センサー制御ユニットから受信した信号を解析し, リズム用 (3 台) およびビート用 (1 台) のスレーブデバイスに向けて制御フラグを一斉送信する.
3. **【楽器制御】楽曲の演奏とテンポ追従:** スレーブデバイスは開始指示を受信した直後から楽曲データの配列を読み込み, 指定された初期テンポで発音を開始する. 演奏中にテンポ変化の指示を受信した場合は, 次回の音符発音時の待機時間を再計算し, 指定の速度へスムーズに移行する.

3.5 テストの設計

本節では、システムの各ユニットが要件を満たしているかを検証するテスト計画について述べる。

3.5.1 センサー制御ユニットのテスト設計

- ・ **閾値と判定テスト**: 指揮棒を振った際の加速度変動が正確に閾値を超え、誤検知なくトリガーフラグが生成されるかを確認する。
- ・ **テンポ変換テスト**: 距離データの変動が正しく BPM 値にマッピングされ、送信バッファに格納されるかを確認する。

3.5.2 同期制御ユニットのテスト設計

- ・ **ルーティングと送信テスト**: センサー制御ユニットから受信した信号が欠損なく解析され、接続された 4 台の楽器ユニットに対して遅延なくブロードキャスト送信されるかをオシロスコープやシリアルモニタを用いて検証する。

3.5.3 楽器制御ユニットのテスト設計

- ・ **役割別演奏テスト**: リズム用ユニットが指定の音階を、ビート用ユニットが指定のタイミングで独立して正確に演奏可能かを確認する。
- ・ **同期応答テスト**: 4 台のユニットが同期制御ユニットからの指示を同時に受信し、全体としてアンサンブルのズレ（ジッタ）が許容範囲内に収まっているかを評価する。
- ・ **テンポ追従テスト**: 演奏中に新たな BPM 指示を受信した際、プログラムが破綻せず直ちに演奏速度が調整されるかを確認する。